

Lab 2 Report: Performance Measurement and Optimization

The motivation for Lab 2 Part Two is to improve the branch-prediction accuracy of the TinyRV pipeline on the `towers.mem` benchmark. In the report below, we will document three design variants and discuss their impact(s).

We observe three predictor parameters (located in `define.vh`) which influence the table below:

- `BHRBITS` (global history width) which sets the PHT depth ($\text{PHT_ENTRIES} = 2^{\text{BHRBITS}}$)
- `BTB_ENTRIES` / `BTB_INDEX_BITS` for BTB capacity
- `USE_PHT_PC_HASH` flag that XORs two PC slices before combining with global history

Label	BHRBITS	BTB_ENTRIES	USE_PHT_PC_HASH	Notes	Accuracy
Baseline	8	16	FALSE	Matches the reference FE-stage predictor	37.03%
Variant A (Larger BTB)	8	32	FALSE	Doubled BTB capacity to cut down direct-mapped conflicts	46.42%
Variant B (Wider GShare)	10	32	FALSE	Increased global history depth	46.22%
Variant C (Best Design)	10	64	TRUE	Adds both wider history and hashed PC slices with a larger BTB	58.22%

Baseline: The starter predictor passes the 30% accuracy threshold but observes significant BTB thrashing since only 16 entries are available for the ~990 branch/jumps found in `towers.mem`.

Variant A: Increasing the BTB to 32 entries boosts accuracy by ~9% without touching PHT. This confirms the previous assumption that capacity/conflict misses were the primary issue in Baseline.

Variant B: Increasing `BHRBITS` to 10 – allowing for a 1024-entry PHT – without increasing the BTB does not provide additional benefit beyond Variant A. The predictor is history-rich but still runs into issues when multiple status branches alias to the same BTB slot.

Variant C: When combining a 64-entry BTB with hashed PC slices, BTB conflicts and PHT aliasing are almost entirely eliminated. We observe the accuracy increases to ~58%, resulting in the best design of the three variants.